A Forward Pass Structure Analysis and Canonical Architecture Mapping of a Code-Based Toy Transformer Encoder-Decoder

Jiwoo Park (jiwoo93@kookmin.ac.kr)
Department of Artificial Intelligence Applications
Graduate School of Software Convergence, Kookmin University
Seoul, Republic of Korea
https://scrc.kookmin.ac.kr/scrc/index.do

***Abstract*—*This paper presents a systematic, code-level analysis of the forward pass of a small-scale C++ Toy Transformer Encoder-Decoder and maps its execution to the canonical architecture of Vaswani et al. The implementation comprises main.cpp, forward.cpp, and forward.h, and operates with manually configured toy weights* ($d_{model} = 4, \quad n_{heads} = 2, \quad |V| = 5$) *instead of trained parameters.*

Three principal findings are established: (1) the code executes a single Encoder block and a single Decoder block (Nx = 1), constituting a pedagogical single-unroll canonical block demonstration; (2) encoderOutput serves exclusively as Key and Value in the Decoder's Cross-Attention sublayer and is not injected into the Masked Self-Attention; and (3) the Decoder's precise execution order is empirically verified at the code level as Masked Self-Attention → Add & Norm → Cross-Attention → Add & Norm → FFN → Add & Norm. All 13 forward-pass steps are traced numerically and interpreted. Deviations from the canonical architecture are identified, and a concrete Nx-extension design is proposed.

***Index Terms*—*Transformer, Encoder-Decoder, Forward Pass, Multi-Head Attention, Cross-Attention, Masked Self-Attention, Layer Normalization, Positional Encoding.*

I. INTRODUCTION

The Transformer architecture, introduced by Vaswani et al. in "Attention Is All You Need" [1], has become the de facto standard across natural language processing, computer vision, speech recognition, and protein structure prediction. Its key innovation is the ability to capture global sequence dependencies through attention alone, without recurrence or convolution.

Despite widespread use, the exact code-level routing of intermediate representations within the Encoder-Decoder structure is frequently misunderstood. In particular, early-stage researchers often conflate the two Decoder attention sublayers: Masked Multi-Head Self-Attention and Encoder-Decoder Cross-Attention. The precise point at which the Encoder Memory is injected into the Decoder is not always obvious from architecture diagrams alone.

This paper addresses that gap by analyzing a minimal C++ Toy Transformer implementation at the code and numerical level. Three concrete goals are pursued: (i) stepwise decomposition and canonical mapping of the full 13-step forward pass; (ii) identification of architectural deviations from the canonical model; and (iii) a concrete design for Nx-layer extension. Analysis is restricted to the forward pass; backpropagation and training are excluded.

II. THEORETICAL BACKGROUND

A. Canonical Transformer Architecture

The canonical Transformer [1] encodes a source sequence $x = (x_1, \ldots, x_n)$ into continuous representations $z = (z_1, \ldots, z_n)$ and autoregressively decodes an output sequence $y = (y_1, \ldots, y_m)$. The Base Model uses $d_{\text{model}} = 512$, $h = 8$, $d_{ff} = 2048$, and $N = 6$ identical layers in both Encoder and Decoder. The toy implementation uses $d_{\text{model}} = 4$, $h = 2$, $d_{ff} = 8$, $N = 1$.

B. Multi-Head Attention

Scaled Dot-Product Attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

where $\sqrt{d_k}$ prevents gradient vanishing when dot products are large. Multi-Head Attention applies h parallel projections and concatenates results:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \quad (2)$$

$$\text{head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right) \quad (3)$$

Within the Transformer, attention operates in three distinct modes: (1) Encoder Self-Attention, Q = K = V = encoder input; (2) Decoder Masked Self-Attention, Q = K = V = decoder input with causal masking; (3) Encoder-Decoder Cross-Attention, Q from decoder, K = V from encoder output.

C. Position-wise Feed-Forward Network

The FFN applied after each attention sublayer is:

$$FFN(x) = \max(0, x W_1 + b_1) W_2 + b_2 \quad (4)$$

The toy implementation uses a 4 → 8 → 4 expansion (4× ratio), consistent with the canonical $d_{ff} = 4\ x\ d_{model}$ convention.

D. Layer Normalization and Residual Connection

Each sublayer output is processed by:

$$Output = LayerNorm(x + Sublayer(x)) \quad (5)$$

$$LayerNorm(u) = \gamma \odot (u - \mu)/\sqrt{(\sigma^2 + \varepsilon)} + \beta \quad (6)$$

where μ and σ² are computed along the feature dimension of a single sample, γ and β are learnable affine parameters, and $\varepsilon = 10^{-5}$. The toy implementation omits γ and β (Affine-Free Simplified LayerNorm).

E. Sinusoidal Positional Encoding

$$PE(pos, 2i) = sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (7)$$

$$PE(pos, 2i+1) = cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (8)$$

Positional encoding is added element-wise to the token embeddings before any attention.

III. IMPLEMENTATION ANALYSIS

A. File Structure and Data Types

The implementation is divided across three files. forward.h declares Vec, Matrix, AttentionWeights, and FFNWeights along with function prototypes. forward.cpp implements the numerical kernels: tokenEmbedding, positionalEncoding, softmax, matMul, linear, layerNormOne, scaledDotAttention, multiHeadAttention, feedForwardBlock, addNorm, and argmax. main.cpp defines all toy weights and executes the full 13-step forward pass serially.

B. Experimental Configuration

Hyperparameters are intentionally minimized for numerical traceability. dModel = 4, numHeads = 2, |V| = 5 (vocabulary: <sos>, I, love, transformers, <eos>). The per-head dimension is $d_k = 2$, giving attention scaling factor √2. Encoder input tokens = {1, 2} ("I love"); Decoder input tokens = {0, 3} ("<sos> transformers", teacher-forcing). All weight matrices are manually set toy values designed to produce numerically stable, interpretable outputs culminating in <eos> prediction.

TABLE I

HYPERPARAMETER COMPARISON: CANONICAL VS. TOY

Hyperparameter	*Canonical Base*	*Toy*
d_model	*512*	*4*
h (heads)	*8*	*2*
d_ff	*2048*	*8*
N (layers)	*6*	*1*
|V|	*~37 000*	*5*

C. Complete Forward Pass Flow

main.cpp serializes the forward pass across 13 steps organized into three streams: Encoder Stream (STEPs 1–6), Decoder Stream (STEPs 7–11), and Prediction Stream (STEPs 12–13). Table II maps each step to its canonical architecture block.

TABLE II

STEP-TO-ARCHITECTURE MAPPING

STEP	*Architecture Block*	*Key Variable*
1	*Inputs*	*encoderTokenIds, decoderTokenIds*
2	*Embedding Lookup*	*tokenEmbedding, embeddingTable*
3	*Positional Encoding*	*positionalEncoding, addMatrix*
4	*Enc Self-Attention*	*multiHeadAttention(..., causal=F)*
5	*Enc Add & Norm 1*	*addNorm(encIn, encAttn)*
6	*Enc FFN + Add&Norm 2*	*feedForwardBlock → encoderOutput*
7	*Dec Masked Self-Attn*	*multiHeadAttention(..., causal=T)*
8	*Dec Add & Norm 1*	*addNorm(decIn, maskedAttn)*
9	*Dec Cross-Attention*	*multiHeadAttention(decNorm1, encOut, ...)*
10	*Dec Add & Norm 2*	*addNorm(decNorm1, crossAttn)*
11	*Dec FFN + Add&Norm 3*	*feedForwardBlock → decoderOutput*
12	*Linear Projection*	*decoderOutput.back() × outputWeight*
13	*Softmax + Argmax*	*softmax, argmax → predicted token*

IV. STEP-BY-STEP NUMERICAL ANALYSIS

A. STEP 1 – Inputs

The source sequence is "I love" (encoder token IDs = [1, 2]) and the teacher-forced decoder input is "<sos> transformers" (decoder token IDs = [0, 3]). The learning objective at inference time is to predict the next token following "transformers" given that the encoder has processed

"I love."

B. STEP 2 – Embedding Lookup

Each integer token ID is projected to a $d_{model} = 4$ dimensional dense vector via the shared embedding table $E \in \mathbb{R}^{|\mathbb{V}| \times d_{model}}$ The resulting Encoder and Decoder embeddings are:

```
Encoder X:  I     = [0.9500, 0.1500, 0.2000, 0.0500]
            love  = [0.2000, 0.8500, 0.2500, 0.1500]
Decoder Y:  <sos> = [0.1200, 0.1000, 0.0800, 0.0500]
            trans = [0.2500, 0.2000, 0.9500, 0.8200]
```

The embedding vectors for "I" and "love" show distinct activation patterns: "I" has a dominant first dimension (0.95) while "love" has a dominant second dimension (0.85), reflecting their manually assigned semantic separation.

C. STEP 3 – Sinusoidal Positional Encoding

Sinusoidal positional encodings are generated and added element-wise:

```
PE(0) = [0.0000, 1.0000, 0.0000, 1.0000]
PE(1) = [0.8415, 0.5403, 0.0100, 0.9999]


Enc X+PE:  I     = [0.9500, 1.1500, 0.2000, 1.0500]
           love  = [1.0415, 1.3903, 0.2600, 1.1500]
Dec Y+PE: <sos> = [0.1200, 1.1000, 0.0800, 1.0500]
          trans = [1.0915, 0.7403, 0.9600, 1.8199]
```

Position 0 has zero contribution on odd dimensions from the sine component, while position 1 adds a large sine value (0.8415) to the first dimension, effectively

differentiating the two sequence positions. The positional signal is larger in magnitude than some embedding dimensions, ensuring that positional information is not overshadowed.

D. STEP 4 – Encoder Self-Attention

Multi-Head Self-Attention is applied with Q = K = V = encoder input, numHeads = 2, causalMask = false. The per-head dimension is $d_k = 2$. Projected Q, K, V and intermediate attention scores are:

```
Q: I    = [1.0225, 1.1025, 0.4275, 0.9025]
   love = [1.1339, 1.3249, 0.5075, 0.9926]
K: I    = [0.7800, 1.1075, 0.5375, 0.9975]
   love = [0.8592, 1.3348, 0.6367, 1.0946]
V: I    = [0.9750, 1.0350, 0.4575, 1.0175]
   love = [1.0818, 1.2424, 0.5575, 1.1185]
```

Head 0 (dims 0–1) attention probabilities for token "I": [0.4417, 0.5583], indicating slight preference toward "love." Head 1 (dims 2–3) probabilities: [0.4770, 0.5230]. Both heads show near-uniform distributions because the toy weight matrices induce only modest score differences. After concatenation and output projection, the encoder attention output is:

```
I    = [1.0404, 1.2495, 0.5404, 0.9888]
love = [1.0416, 1.2516, 0.5407, 0.9890]
```

E. STEP 5 – Encoder Add & Norm 1

The residual sum x + Sublayer(x) and subsequent LayerNorm are computed. For token "I":

$$x_{\text{res}} = [1.9904, 2.3995, 0.7404, 2.0388] \qquad (9)$$

$\mu = (1.9904 + 2.3995 + 0.7404 + 2.0388) / 4 = 1.7923$, $\sigma^2 = 0.3940$, $\sigma = 0.6277$. Normalized output for token "I:"

$$\dot{x} = [0.3158, 0.9676, -1.6762, 0.3928] \qquad (10)$$

LayerNorm centers and scales the residual sum to zero mean and unit variance, preventing feature scale explosion across sublayers. Token "love" yields $\dot{x}$ = [0.2455, 1.0676, −1.6409, 0.3278], reflecting similar structural normalization.

F. STEP 6 – Encoder FFN and Add & Norm 2

The FFN applies a 4 → 8 expansion with ReLU, then projects back to dimension 4. Pre-ReLU hidden activations for token "I":

```
h_pre  = [ 0.3516,  0.1956, -0.6257,  0.0934,
0.3458, -0.1388, -0.4169,  0.2546]
h_ReLU = [ 0.3516,  0.1956,  0.0000,  0.0934,
0.3458,  0.0000,  0.0000,  0.2546]
FFN out = [0.2195,  0.1455,  0.0969,  0.2126]
```

Three of eight hidden units are gated to zero by ReLU (indices 2, 5, 6), introducing nonlinear sparsity. After the second Add & Norm, the final encoderOutput (Encoder Memory) is:

```
I    = [0.3548,  0.9139, -1.6913,  0.4226]
love = [0.2723,  1.0272, -1.6563,  0.3567]
```

Both tokens produce strongly negative third-dimension values (−1.69, −1.66), reflecting the pattern established through self-attention and normalization. This encoderOutput will serve exclusively as Key and Value in the Decoder's Cross-Attention (STEP 9).

G. STEP 7 – Decoder Masked Self-Attention

Masked Self-Attention is applied to the Decoder input with causalMask = true. At position 0 (<sos>), future position 1 is masked with score = -1×10^9, forcing the softmax to assign probability ≈ 1.0 to position 0 itself. Head 0 probabilities for <sos>: [1.0000, 0.0000]; for "transformers": [0.3536, 0.6464]. Head 1 probabilities for <sos>: [1.0000, 0.0000]; for "transformers": [0.1449, 0.8551]. After output projection:

```
<sos>  = [0.3067, 0.9241, 0.4021, 0.9262]
trans  = [0.7851, 1.0200, 1.0770, 1.5786]
```

The causal mask is the operationally critical distinction between Decoder Self-Attention and Encoder Self-Attention. Its purpose is to enforce autoregressive generation: each output position may only attend to positions up to and including itself.

H. STEP 8 – Decoder Add & Norm 1

Residual addition and LayerNorm are applied after Masked Self-Attention:

```
<sos>  residual = [0.4267, 2.0241, 0.4821, 1.9762]
       normed   = [-1.0353, 1.0304, -0.9636, 0.9685]
trans  residual = [1.8766, 1.7603, 2.0370, 3.3986]
       normed   = [-0.5932, -0.7694, -0.3501,
1.7127]
```

The normalized Decoder state (decoderNorm1) is fed as Query into the Cross-Attention sublayer in the next step.

I. STEP 9 – Decoder Cross-Attention (Critical Routing Point)

This step is the architectural focal point of the paper. The routing of Q, K, V is:

$$Q = \text{linear}(\text{decoderNorm1}, W_Q) \qquad (11)$$

$$K = \text{linear}(\text{encoderOutput}, W_K) \qquad (12)$$

$$V = \text{linear}(\text{encoderOutput}, W_V) \qquad (13)$$

Encoder output is therefore injected only here, as Key and Value. It is not present in the Masked Self-Attention of STEP 7. This confirms that the encoder-decoder information interface is exclusively the Cross-Attention sublayer, consistent with the canonical paper. Numerical values:

```
K (from encoderOutput):
  I    = [ 0.0738,  0.6885, -1.2022,  0.3130]
  love = [ 0.0137,  0.7937, -1.1845,  0.2472]
Cross-attention output:
  <sos>  = [0.3838,  0.4412, -1.0798,  0.1683]
  trans  = [0.3855,  0.4388, -1.0801,  0.1687]
```

Attention probabilities for <sos> token, head 0: [0.4774, 0.5226] over [I, love], indicating roughly equal attention

to both source tokens. This is consistent with the fact that neither source token has been dramatically distinguished by the toy weights.

J. STEP 10 – Decoder Add & Norm 2

Residual addition fuses the Cross-Attention context with the Decoder state:

```
<sos>  normed = [-0.4438,  1.0522, -1.4246,  0.8162]
trans  normed = [-0.1553, -0.2579, -1.1762,  1.5893]
```

The fourth dimension of "transformers" position increases substantially (1.5893), reflecting accumulated context from both self-attention and cross-attention.

K. STEP 11 – Decoder FFN and Add & Norm 3

The Decoder FFN applies the same 4→8→4 structure with ReLU:

```
trans h_pre  = [ 0.2938, -0.3476, -0.4450,  0.5144,
0.4452, -0.4319, -0.1939,  0.5010]
trans h_ReLU = [ 0.2938,  0.0000,  0.0000,  0.5144,
0.4452,  0.0000,  0.0000,  0.5010]
FFN out      = [ 0.2237,  0.1465,  0.1864,  0.4043]
```

Four of eight hidden units are zeroed by ReLU. Final decoderOutput (decoderOutput.back(), i.e., the "transformers" position):

$$h_{last} = [-0.1578, -0.3230, -1.1299, 1.6106] \quad (14)$$

The large positive fourth component and the strong negative third component are decisive in steering the vocabulary projection toward <eos>.

L. STEP 12–13 – Linear Projection, Softmax, and Prediction

The final Decoder hidden state is projected to vocabulary logits:

$$z = h_{\text{last}} \times W_{\text{vocab}} \in \mathbb{R}^{|V|} \quad (15)$$

```
h_last = [-0.1578, -0.3230, -1.1299,  1.6106]
logits = [-0.0545, -0.0963, -0.1097, -0.1859,
0.9488]

Softmax probabilities:
  <sos>        : 0.153618
  I            : 0.147333
  love         : 0.145374
  transformers : 0.134701
  <eos>        : 0.418973   <- predicted
```

The <eos> logit (0.9488) is substantially higher than all others, yielding a softmax probability of 0.419. This arises because the outputWeight column for <eos> strongly amplifies the large positive fourth dimension of h_last (W[3][4] = 0.85). The predicted next token is therefore <eos>, which is semantically appropriate: having processed "I love" on the Encoder side and "<sos> transformers" on the Decoder side, the model predicts end-of-sentence.

The vocabulary probability ordering is <eos> (0.419) > <sos> (0.154) > I (0.147) > love (0.145) > transformers (0.135). The four non-<eos> tokens have close probabilities because their logits (−0.05 to −0.19) differ by less than 0.14, whereas <eos> logit (0.95) exceeds all others by more than 1.0 in absolute terms.

V. DEVIATIONS FROM THE CANONICAL ARCHITECTURE

A. Affine-Free Simplified LayerNorm

The layerNormOne function computes $\hat{x} = \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}}$ *without learnable γ and β. This is Affine-Free Simplified LayerNorm; the canonical form additionally multiplies and shifts by learned scale and bias parameters. Numerically, the toy implementation behaves as* if $\gamma = 1$ and $\beta = 0$, *i.e., the post-normalization identity affine transformation.*

B. Weight Sharing Policy

The canonical Transformer ties the input embedding, output embedding, and pre-softmax projection matrix [6]. The toy implementation shares the embeddingTable between Encoder and Decoder token lookup but uses a separate outputWeight for the final vocabulary projection. Its policy is therefore Embedding-Shared, Pre-Softmax-Unshared.

C. Absence of Nx Stacking

The canonical Base Model stacks N = 6 identical layers. The toy code executes each block (STEPs 4–6 and 7–11) only once, yielding Encoder depth = Decoder depth = 1. This is not caused by misunderstanding numHeads: the inner loop in forward.cpp iterates over attention heads (numHeads = 2), not over layer depth. The omission of Nx is an intentional pedagogical simplification for single-pass numerical traceability.

VI. NX-LAYER EXTENSION DESIGN

A. Structural Conditions

Three conditions must hold for Nx stacking: (i) all layer input/output shapes are fixed at $[seq_{len} \times d_{\text{model}}]$*; (ii) encoderOutput is shared as memory across all Decoder Cross-Attention layers; and (iii) each layer holds independently initialized parameter instances.*

B. Recommended Design

The extension requires two new struct types: EncoderLayerWeights (selfAttn, ffn) and DecoderLayerWeights (selfAttn, crossAttn, ffn). Corresponding runEncoderLayer and runDecoderLayer functions wrap the existing kernels without modifying them. In main.cpp, ***std::vector*** *containers of size Nx hold per-layer weights, and two for-loops execute the Encoder and Decoder stacks respectively. The key conclusion is that all existing numerical kernels are correct and reusable; only the stack orchestration layer is absent.*

VII. CONCLUSION

This paper analyzed a C++ Toy Transformer Encoder-Decoder at the code and numerical level and mapped its full 13-step forward pass to the canonical architecture. Three principal conclusions were established. First, the implementation is a pedagogically valid Nx = 1 single-unroll canonical block demonstration whose core kernels are consistent with the canonical mathematical formulation. Second, encoderOutput is injected exclusively into the Decoder Cross-Attention as Key and Value; the precise verified Decoder execution order is Masked Self-Attention → Add & Norm → Cross-Attention → Add & Norm → FFN → Add & Norm. Third, three intentional simplifications are present relative to the canonical architecture: Affine-Free LayerNorm, Embedding-Shared/Pre-Softmax-Unshared weight policy, and the absence of Nx stacking. Numerical tracing confirms that these simplifications yield consistent, interpretable results, culminating in a correct <eos> prediction (P = 0.419) for the toy example. The codebase is in a state where stack orchestration alone remains as future work, not a complete reimplementation.

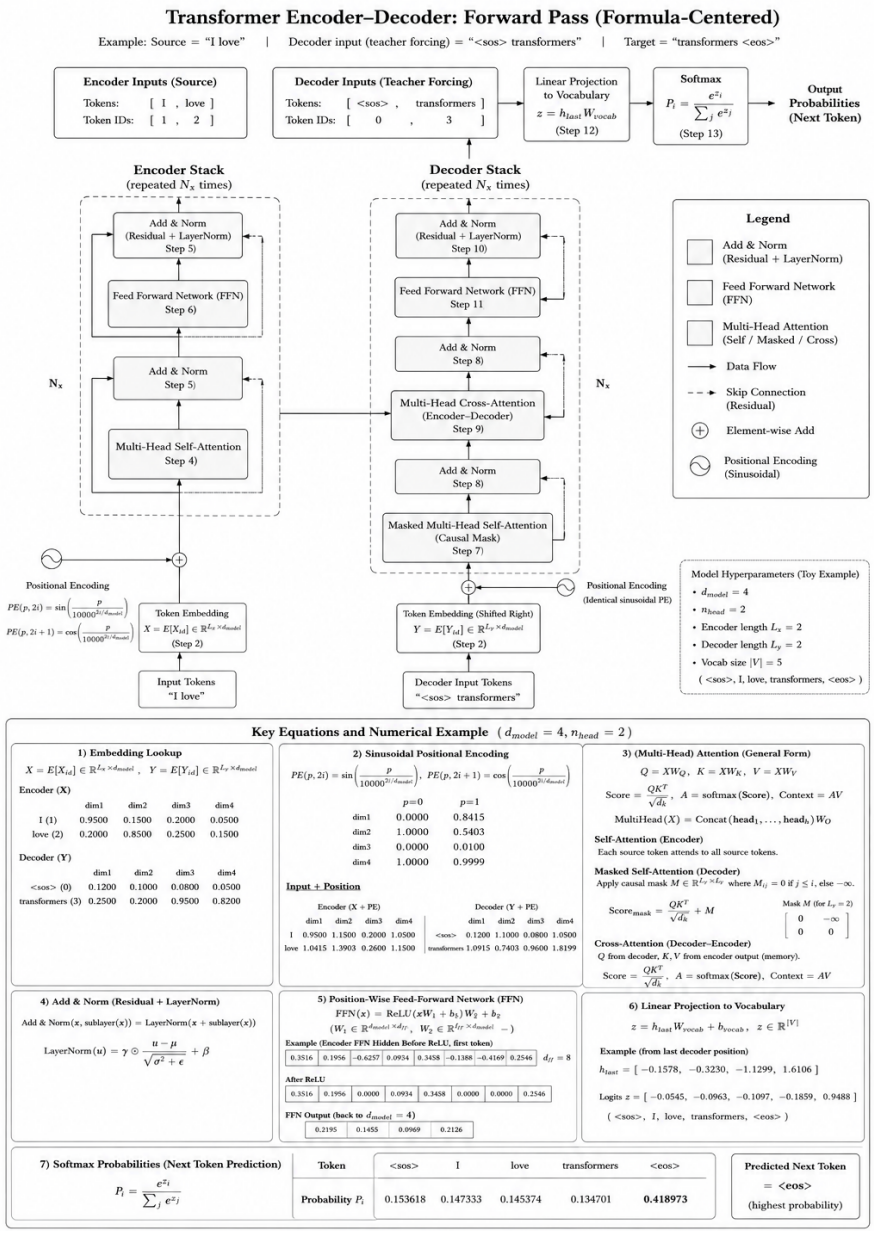

Fig. 1. *Formula-centered overview of the Transformer Encoder-Decoder forward pass for the toy example (source = "I love," decoder input = "<sos> transformers"). The figure summarizes embedding lookup, sinusoidal positional encoding, encoder self-attention, decoder masked self-attention, encoder-decoder cross-attention, residual Add & Norm operations, position-wise FFN, and the final softmax prediction stage. Using the logged numerical outputs, the last decoder state yields logit 0.9488 for <eos>, corresponding to the highest output probability (0.419).*

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in Adv. Neural Inf. Process. Syst., vol. 30, pp. 5998–6008, 2017.

[2] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv preprint arXiv:1607.06450, 2016.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI Blog, vol. 1, no. 8, p. 9, 2019.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. CVPR, 2016, pp. 770–778.

[6] O. Press and L. Wolf, "Using the output embedding to improve language models," in Proc. EACL, 2017, pp. 157–163.